

Lab Report No _1_

Microprocessor and Interfacing techniques



Submitted By:

Huzaifa Shahbaz

21-CE-009

Submitted to:

Engr. Bushra Fiaz

Dated:

Week 01

**Department of Computer Engineering,
HITEC University, Taxila**

Lab Task No 1:

Solution:

Brief description (3-5 lines)

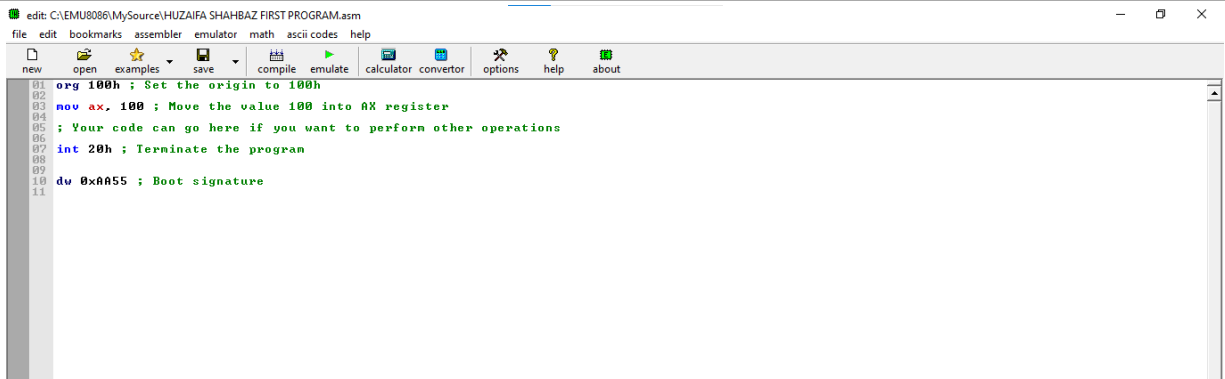
1. Write down the following code in emulator 8086:

```
01 org 100h
02 mov ax,100
```

Run the code step by step and observe the value changing. Answer the following questions:

- What is the value stored in register AX ?
- What are the value stored in AL and AH ?
- The values stored in AX is NOT hundred, Why?

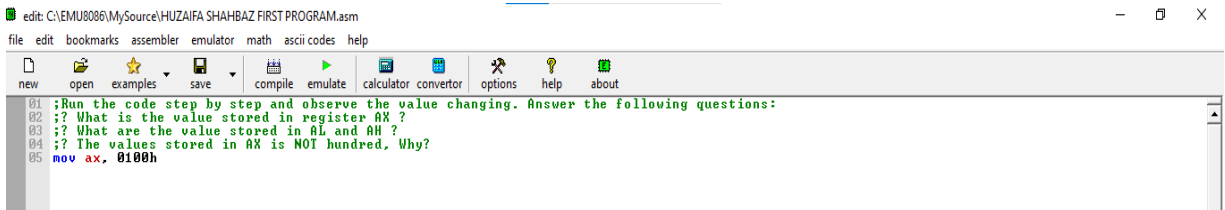
The code



The screenshot shows the 8086 emulator window with the following assembly code:

```
01 org 100h ; Set the origin to 100h
02
03 mov ax, 100 ; Move the value 100 into AX register
04
05 ; Your code can go here if you want to perform other operations
06
07 int 20h ; Terminate the program
08
09
10 dw 0x0A55 ; Boot signature
11
```

- What is the value stored in register AX ?
- What are the value stored in AL and AH ?
- The values stored in AX is NOT hundred, Why?



The screenshot shows the 8086 emulator window with the following instructions:

```
01 ;Run the code step by step and observe the value changing. Answer the following questions:
02 ;? What is the value stored in register AX ?
03 ;? What are the value stored in AL and AH ?
04 ;? The values stored in AX is NOT hundred, Why?
05 mov ax, 0100h
```

The results (Screenshot)



ANSWER:

- 1: The value stored in register AX is 100 in decimal or 64th (in hexadecimal).
- 2: The value stored in register AL is 64th (in hexadecimal) or 100 (in decimal) and the value stored in AH is 0.
- 3: The value stored in AX is not “hundred” because in assembly language, numerical values are represented in a binary format and the instruction ‘mov ax,100’ is used to move the decimal number 100 (or 64h) in hexadecimal into the AX register. The computer doesn’t interpret the value as a text string; it treats it as a numeric value.

Lab Task No 2:

Solution:

Brief description (3-5 lines)

1. Type the following code in your emulator 8086 environments:

```

01 .model small
02 .data ; Define variables in this portion
03 a db 15
04 b db 11
05 sum1 db 0
06
07 .code ; Write the normal code here
08
09 ; For now just add these two lines
10 ; The reason will be explained later
11 mov ax,@data
12 mov ds,ax
13
14
15 ; Now summing up two values
16 mov ah,a
17 mov al,b
18 add al,ah ; now al holds the sum
19
20 mov sum1,al

```

Run the code step by step and observe the value(s) changing. Answer the following questions:

1: Write down the changes being occurred in each register with each step execution and tell why the values change (for each register).

2: Locate the values stored in memory of the processor and observe changes before and after execution of instruction.

3: Show the values stored (in form of variables)

The code

Answer:

1: Program Counter (PC):

It increments to point to the next instruction in memory to ensure sequential execution.

Instruction Register (IR):

It stores the current instruction being executed, updating as the PC advances.

Accumulator (ACC):

Values change based on arithmetic or logical operations specified by the instruction.

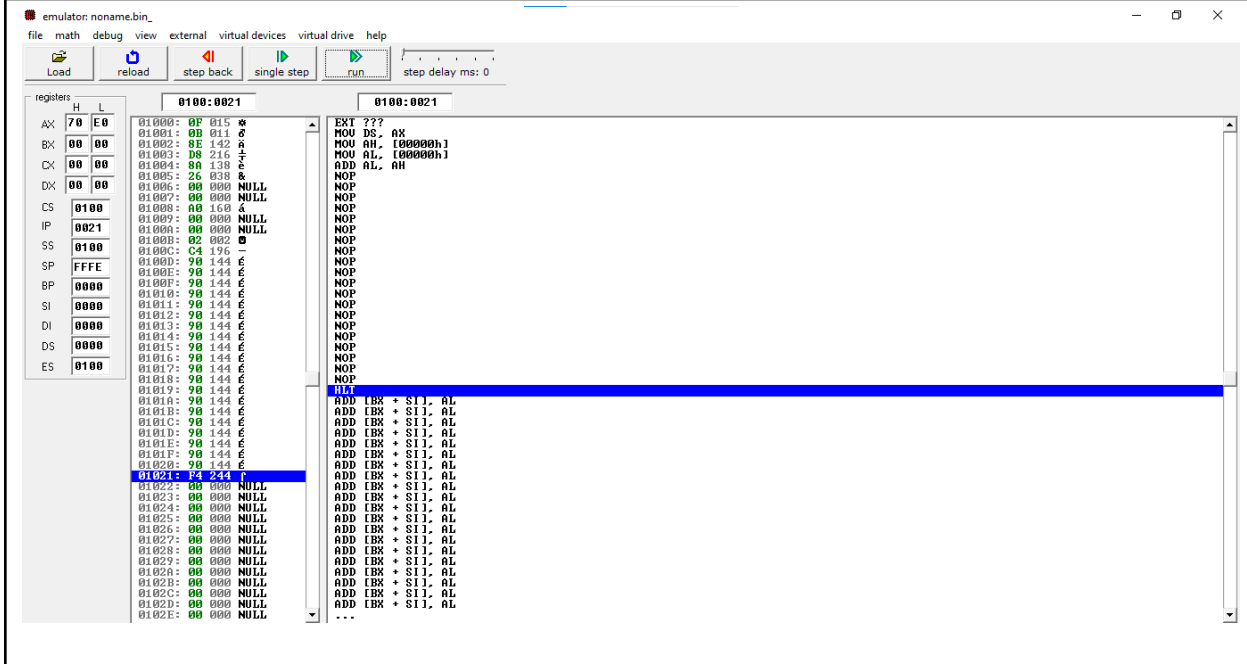
Memory Address Register (MAR):

Holds the address of data being accessed; changes when loading or storing data.

Memory Buffer Register (MBR):

Stores data being transferred between memory and CPU, changing during data load/store operations.

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

Write a Program that takes 10 numbers one by one from memory and adds them in a variable. In the end store the output in a new location in memory.

i:Task 3 reports is evaluated in report

The code

```

01 org 100h
02
03 numbers db 10 ; Define an array to store 10 numbers
04 result db 0 ; Define a variable to store the result (16-bit)
05
06
07 mov cx, 10 ; Set the loop counter to 10 (number of inputs)
08 mov si, 0 ; Initialize source index (index for numbers array)
09 mov bx, 0 ; Initialize sum to zero
10
11
12 input_loop:
13 mov ah, 01h ; Function to read a character from standard input
14 int 21h ; DOS interrupt for input
15 sub al, '0' ; Convert ASCII character to numeric value
16 mov [numbers + si], al ; Store the input number in the array
17 inc si ; Move to the next position in the array
18
19 dec cx ; Decrement the loop counter
20 jnz input_loop ; Continue the loop if CX is not zero
21
22 mov cx, 10 ; Reset the loop counter for summing
23 mov si, 0 ; Reset source index
24
25 sum_loop:
26 mov al, [numbers + si] ; Load a number from the array into AL
27 add bx, ax ; Add the number to the sum in BX
28 inc si ; Move to the next position in the array
29
30 dec cx ; Decrement the loop counter
31 jnz sum_loop ; Continue the loop if CX is not zero
32
33 mov [result], bx ; Store the result in the result variable
34
35 ; You can add code here to store the result in a new location in memory
36 ; For example, if you want to store it at memory address 0x8000:
37 ; mov [0x8000], bx
38
39 ; Terminate the program
40 mov ah, 4Ch
41 int 21h
42
43
44
45 ret

```

The results (Screenshot)

emulator: noname.com

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers		0700:0116	0700:0116
H	L		
AX	01 08	07110: 2C 044	SUB AL, 030h
BX	00 00	07111: 30 048	MOV [SI] + 00100h, AL
CX	00 2D	07112: 88 136	INC SI
DX	00 00	07113: 84 132	DEC CX
CS	0700	07114: 00 000 NULL	JNE 020Ch
IP	0116	07115: 01 001	MOV CX, 00000h
SS	0700	07116: 46 020	MOV SI, 00000h
SP	FFFF	07117: 49 073	MOV AL, [SI] + 00100h
BP	0000	07118: 75 117	ADD BX, AX
SI	0000	07119: F2 242	INC SI
DI	0000	0711A: B9 185	DEC CX
DS	0700	0711B: 0A 010 NEVL	JNE 0120h
ES	0700	0711C: 00 000 NULL	MOV [00101h], BX
		0711D: BE 198	MOV AH, 04Ch
		0711E: 00 000 NULL	INT 021h
		0711F: 00 000 NULL	RET
		07120: 8A 138	NOP
		07121: 84 132	NOP
		07122: 00 000 NULL	NOP
		07123: 01 001	NOP
		07124: 03 003	NOP
		07125: D8 216	NOP
		07126: 46 070	NOP
		07127: 49 073	NOP
		07128: 75 117	NOP
		07129: F6 246	NOP
		0712A: B9 137	NOP
		0712B: 1E 030	NOP
		0712C: 01 001	NOP